Advanced Programming for Scientists

Lecture 5

Versioning II: Collaboration and CI/CD

v1.0

Prof. Mateusz Bawaj

A.D. 1308 unipg
UNIVERSITÀ DEGLI STUDI DI PERUGIA

Collaboration Matters in Science

Modern science is collaborative by design

- Code is part of the scientific result
- Reproducibility requires traceability
- Reviewability requires structured workflows
- **Automation reduces human error**

Examples:

- Multi-author simulation frameworks
- Shared ML pipelines
- Software in large collaborations

From Local Git to Collaborative Git

Recap from Lesson 2:

- `git init`, `add`, `commit`
- History as a directed graph
- Automatic changelog via commit messages

Today:

- Remote repositories
- Branching models
- Pull requests
- Code review
- CI pipelines

Distributed Model Refresher

Git is distributed:

- Every clone contains full history
- Offline commits possible
- Synchronization via remotes

Typical remote service providers:

- GitHub
- GitLab
- Institutional servers

Scientific implication:

- Long-term preservation of research code
- Forks enable methodological variations

Repository Roles in a Scientific Project

In a research-oriented repository:

- `main` → stable, reproducible state
- `develop` → integration branch
- `feature/*` → new functionality
- `experiment/*` → risky or exploratory code

Example:

- `feature/gpu-acceleration`
- `experiment/new-noise-model`

Pull Requests as Scientific Review

Pull Request (PR):

- Structured discussion
- Inline code comments
- Automated checks
- Traceable decisions

Analogy: PR ≈ internal peer review before publication

Best practice:

- Small PRs
- Clear description
- Link to issue
- Reference benchmark results

Writing Meaningful Commit Messages

Bad:

- "fixed stuff"

Good:

- "Implement time-domain recursion with truncated memory buffer"

Structure:

- Short summary
- Motivation
- Technical change
- Impact on performance or results

Conflict Resolution

Conflicts are normal

Typical causes:

- Same file modified
- Same function edited differently
- Reformatting conflicts

Scientific advice:

- Keep functions small
- Separate numerical core from I/O and from user interface
- Avoid large monolithic files

Conflict Resolution

```
git fetch origin
```

```
git checkout my-branch-with-conflict
```

`git status` – see conflicted files

Inside conflicted files you'll see markers

```
<<<<<<< HEAD
your code
=======
incoming code
>>>>>>> main
```

Edit the file to keep the correct code and **remove the markers**.

```
git add changed-file
```

```
git commit
```

```
git push origin my-branch-with-conflict
```

Continuous Integration (CI)

CI = automated testing at each push

Typical pipeline:

1) Install dependencies
2) Run unit tests
3) Run benchmarks
4) Build documentation
5) Check formatting

Scientific relevance:

- Detect errors early
- Benchmark your code and results

Example: CI for a Time-Domain Simulator

Pipeline stages:

- Linting (PEP8 compliance)
- Unit tests on recursion
- Compare against reference dataset
- Benchmark for *N* scaling

  e.g. if performance drops by >10% → fail pipeline

Continuous Deployment (CD)

CD extends CI:

- Automatic packaging
- Automatic publication
- Documentation deployment

Examples:

- Publish Python package
- Build documentation website
- Generate API docs

Scientific impact:

- Faster dissemination
- Standardized interfaces

Minimal GitHub Actions Example

```yaml
name: my CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"
      - name: Install dependencies
        run: python -m pip install --upgrade pip pyyaml
      - run: python labs/YAML/python/pyyaml_test.py
```

Key idea:
Automation replaces manual verification.

Key Takeaways: Continuous Integration with GitHub Actions

Continuous Integration (CI)

- CI automatically builds and tests code after every push or pull request
- Implemented via workflow files written in YAML
- Ensures that new commits do not break the project

Practical effect in GitHub

- Each commit triggers the CI workflow
- GitHub shows the result next to the commit (Indicators in the repository history and the number of executed checks)

Homework

Write one Action for your branch.

Take care to commit the action to your branch.

Verify it does work.